

# Eigene Klasse & Objekterzeugung

## Pseudocode (IHK-Stil):

```
klasse Kunde
  attribute: name: String, alter: Integer
  konstruktor(name, alter)
    this.name = name
    this.alter = alter
  methode getName() → String
    return name
  methode setName(n: String)
    name = n
end klasse
```

```
obj = neues Kunde("Max", 25)
```

## Java-Beispiel:

```
class Kunde {
    private String name;
    private int alter;

    public Kunde(String name, int alter) {
        this.name = name;
        this.alter = alter;
    }

    public String getName() { return name; }
    public void setName(String n) { this.name = n; }
}

public class Demo {
    public static void main(String[] args) {
        Kunde k = new Kunde("Max", 25);
        System.out.println(k.getName());
    }
}
```

# Eigene Klassen & Objekte – Erklärung & Beispiele

Eine Klasse ist ein **Bauplan** für Objekte. Attribute (Eigenschaften), Methoden (Funktionalität).  
In Java: Felder oft `private`, Zugriff über Getter/Setter (Kapselung).

**Typische Fehlerquellen:**

- Konstruktor nicht definiert → Standardkonstruktor wird genutzt
- Felder ohne `this` überschrieben
- Getter/Setter vergessen

**Prüfungstipp:** Klassenaufgaben sind Standard in IHK-Prüfungen – sauber strukturieren!

| Element     | Beschreibung                    |
|-------------|---------------------------------|
| Attribute   | Datenfelder (z. B. Name, Alter) |
| Konstruktor | Initialisiert das Objekt        |
| Methoden    | Operationen auf den Feldern     |

# Vererbung & Polymorphie

## Pseudocode (IHK-Stil):

```
klasse Tier
    methode sprich()
        ausgabe("Ich bin ein Tier")
```

```
klasse Hund erweitert Tier
    methode sprich()
        ausgabe("Wuff!")
```

```
obj1 = neues Tier()
obj2 = neues Hund()
obj1.sprich() // Ich bin ein Tier
obj2.sprich() // Wuff!
```

## Java-Beispiel:

```
class Tier {
    public void sprich() {
        System.out.println("Ich bin ein Tier");
    }
}

class Hund extends Tier {
    @Override
    public void sprich() {
        System.out.println("Wuff!");
    }
}

public class Demo {
    public static void main(String[] args) {
        Tier t = new Tier();
        Hund h = new Hund();
        t.sprich(); // Ich bin ein Tier
        h.sprich(); // Wuff!
    }
}
```

# Vererbung & Polymorphie – Erklärung & Beispiele

Vererbung: ``extends`` → Eine Klasse erbt Attribute/Methoden von einer anderen.

Polymorphie: Ein Objekt verhält sich abhängig von seiner konkreten Klasse unterschiedlich.

**\*\*Vorteile:\*\***

- Code-Wiederverwendung
- Einheitliche Schnittstellen
- Möglichkeit zur Überschreibung von Methoden (``@Override``)

**\*\*Typische Fehlerquellen:\*\***

- Überschreiben vergessen oder Signatur falsch
- ``super``-Aufruf im Konstruktor übersehen
- Falscher Zugriffsschutz (private statt protected)

**\*\*Prüfungstipp:\*\*** Polymorphie-Fragen kommen oft mit ``Tier/Hund/Katze``-Beispielen.

| Beispiel      | Ausgabe          |
|---------------|------------------|
| Tier.sprich() | Ich bin ein Tier |
| Hund.sprich() | Wuff!            |

## Interfaces (z. B. Comparable)

### Pseudocode (IHK-Stil):

```
interface Vergleichbar<T>
    methode vergleicheMit(other: T) → Integer

klasse Artikel implementiert Vergleichbar<Artikel>
    attribute: preis: Integer
    methode vergleicheMit(other: Artikel) → Integer
        wenn preis == other.preis dann return 0
        sonst wenn preis > other.preis dann return +1
        sonst return -1
```

### Java-Beispiel:

```
interface Vergleichbar<T> {
    int vergleicheMit(T other);
}

class Artikel implements Vergleichbar<Artikel> {
    int preis;
    Artikel(int p) { this.preis = p; }

    @Override
    public int vergleicheMit(Artikel other) {
        if (this.preis == other.preis) return 0;
        else if (this.preis > other.preis) return 1;
        else return -1;
    }
}
```

# Interfaces – Erklärung & Beispiele

Ein Interface ist ein **Vertrag**: Es definiert Methoden, die eine Klasse implementieren muss.  
In Java: `interface` + `implements`. Mehrfachvererbung ist über Interfaces möglich.

Beispiele:

- Comparable<T> (Sortierordnung festlegen)
- Observer (Muster für Benachrichtigung)

**Typische Fehlerquellen:**

- Methoden nicht implementiert → Compiler-Fehler
- Signaturen abweichend (Parameter, Rückgabe)
- `@Override` weggelassen (führt leicht zu Fehlern)

**Prüfungstipp:** Interface-Aufgaben sind IHK-Klassiker – meist als `Comparable` oder Observer-Pattern.

| Interface  | Nutzen                          |
|------------|---------------------------------|
| Comparable | Objekte vergleichbar machen     |
| Observer   | Benachrichtigung bei Änderungen |
| Runnable   | Nebenläufigkeit starten         |